

EXERCISE I

In reference to the Stochastic Gradient Descent (SDG) algorithm

I.a) Explain the concept of loss function and provide one example.

It's a function that determines the distance between the algorithm's current output and its expected output. It's a way to evaluate how well your algorithm is modeling the data: the lower the loss function, the lower the error of the Neural Network in predicting the right output.

Example of loss function: there are many loss functions, such as mean squared error, cross entropy, exponential.

As an example, the formula of the cross entropy loss function can be used for multi-class classification problems, and its formula is:

$$\text{Loss} = - \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$

- y_i represents the true class label for class 'i' (1 if the true class is 'i', 0 otherwise)
- $\hat{y}_i$ is the predicted probability of class 'i'
- The summation runs over all the classes

Another example: for regression problems, where the goal is to predict continuous numerical values, we can use the Mean squared error (MSE) loss

$$MSE = \frac{1}{n} \sum \left(\underbrace{y - \hat{y}}_{\substack{\text{The square of the difference} \\ \text{between actual and} \\ \text{predicted}}} \right)^2$$

I.b) Why do some loss functions use a regularization term?

Regularization is a technique used in machine learning and deep learning to prevent overfitting and improve the generalization performance of a model. It involves adding a penalty term to the loss function during training. This penalty discourages the model from becoming too complex or having large parameter values, which helps in controlling the model's ability to fit noise in the training data.

I.c) Is SGD guaranteed to minimize the loss function? Motivate your answer.

Stochastic gradient descent converges almost surely to a global minimum when the objective function is convex or pseudoconvex, and otherwise converges almost surely to a minimum that can be either local or global.

EXERCISE II

Consider a convolutional neural network layer with the following characteristics: 80 input channels (32x32), convolutional filters size 5x5, 160 output channels. Padding with zeros and stride 1.

II.a) Compute the number of MAC operations needed to compute a single output channel by applying convolution on a single input channel.

$K=5$ (kernel size)

$C=Win=Hin$

$P=2$

Padding gives us $Win=Wout=32$:

$$\text{Output size} = (C + 2P - K)/(S) + 1 = (32 + 2*2 - 5 + 1) / 1 = 32$$

$$\begin{aligned} \text{N}^\circ \text{ MAC (single channel)} &= \\ &= K*K*Cin* Cout*(Wout*Hout) \text{ MAC} = 5*5*1*1*(32*32)=25600 \text{ MAC} \end{aligned}$$

II.b) Compute the total number of MAC operations needed to compute all the output channels in the layer.

$$\begin{aligned} \text{N}^\circ \text{ MAC (all channels)} &= \\ &= K*K*Cin* Cout*(Wout*Hout) \text{ MAC} = 5*5*80*160*(32*32)=327'680'000 \end{aligned}$$

II.c) Compute the number of SUM and number of DIV operations needed to perform average pooling (2x2) on all the output channels.

hypothesis: no padding, stride 2 $\rightarrow Wout = 16$

$$\text{sum_per_pooling window} = (2*2-1) = 3 \text{ SUM}$$

$$\text{div_per_pool_window} = 1 \text{ DIV}$$

$$\begin{aligned} \text{Pool_ops} &= (3 \text{ SUM} + 1 \text{ DIV}) * Wout*Hout * Cout = (3 \text{ SUM} + 1 \text{ DIV}) * 16*16 * 160 = (3 \text{ SUM} + \\ &1 \text{ DIV}) * 40960 = (3 \text{ SUM} + 1 \text{ DIV}) * 40960 = 122880 \text{ SUM} + 40960 \text{ DIV} \end{aligned}$$

EXERCISE III

A parallel cluster of RISC-V processors is executing a convolutional layer with 9 input channels and 17 output channels. Input spatial size is 128x128. Inputs, outputs, and weights are quantized to 8 bits. Convolutional filter size is 3x3, no padding and stride is used. RELU non-linearity is applied in place on the output. Assume that on average a RISC-V processor can perform a MAC in 2.5 cycles and a RELU operation in 3 cycles. RELU is performed by a single processor in the cluster, while the convolutions are parallelized by assigning each processor to one or more output channels, aiming at balancing execution time, if possible.

III.a) Compute the L1 minimal shared memory size that is needed to store the complete layer (input and output channels as well as weights) in L1.

act_bytes = 1 byte, weight_bytes = 1 byte

Hin = Win = 128 (input spatial size)

Cin = 9

K = 3 (Kernel size)

P = 0 (padding)

Output size = $\text{floor}((W + 2P - K)/(S) + 1) = \text{floor}((128+0-3)/1 + 1) = 126$

Cout = 17

in_size = Cin * Hin * Win = 9 * 128 * 128 = 147456

weights_size = K * K * Cin * Cout = 3 * 3 * 9 * 17 = 1377

out_size = Cout * Hout * Wout = 17 * 126 * 126 = 269892

L1_mem = (In_Size + Out_Size)*Act_bytes + weights_size*weights_bytes = 147456 + 1377 + 269892 = 418725 Bytes

III.b) Compute the total number of cycles to process the layer on an 8-core cluster.

Cycles_conv (8 cores) = (Hout * Wout * Cin * ceil(Cout/Ncores) * K * K) * 2.5 cycles =
= (126 * 126 * 9 * ceil(17/8) * 3 * 3) * 2.5 cycles = (126 * 126 * 9 * 3 * 3 * 3) * 2.5
cycles = 9644670 cycles

Cycles_ReLU(1 core) = (Cout * Wout * Hout) * 3 cycles = (17 * 126 * 126) * 3 cycles =
= 809676 cycles

Cycles_tot = 9644670 + 809676 = 10454346 cycles

III.c) Compute the worst-case percentage of idle time for a core in the cluster in case it's not possible to balance the workload.

Output channels are 17

7 cores calculating 2 channels, and 1 core calculating 3 channels (7*2 + 1*3 = 17)

idle_time_perc = (time (3 ch) - time (2 ch)) / time (3 ch)

$\text{Cycles_conv (1 core)} = (\text{Hout} * \text{Wout} * \text{Cin} * \text{Cout} * K * K) * 2.5 \text{ cycles}$

$\text{Cycles_conv (1 core, 2 out_ch)} = (126 * 126 * 9 * 2 * 3 * 3) * 2.5 \text{ cycles} = 6429780 \text{ cycles}$

$\text{Cycles_conv (1 core, 3 out_ch)} = (126 * 126 * 9 * 3 * 3 * 3) * 2.5 \text{ cycles} = 9644670 \text{ cycles}$

$\text{idle_time_perc} = (9644670 - 6429780) / 9644670 = 33\%$